```python
import customtkinter as ctk
from tkinter import ttk, messagebox, Canvas, filedialog
import json
import os
import csv

ctk.set_appearance_mode("light")
ctk.set_default_color_theme("blue")
FILE_NAME = "inventory_data.json"
 CONFIG_FILE = "config.json"

 class InventoryApp(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Smart Inventory Management System")
        self.geometry("1400x900")
        self.configure(fg_color="#ECECEC")
        self.inventory = {}
        self.categories = set()
        self.low_stock_threshold = 10
        self.selected_ids = []
        self.load_data()
        self.load_config()
        self.create_header()
        self.create_summary()
        self.create_dashboard_stats()
        self.create_dashboard()
        self.create_footer()
        self.update_summary()
        self.protocol("WM_DELETE_WINDOW", self.on_close)
        self.bind("<Control-n>", lambda e: self.add_product_window())
        self.bind("<Control-e>", lambda e: self.export_inventory_shortcut())
        self.bind("<Control-f>", lambda e: self.search_product_window())
        self.bind("<Control-i>", lambda e: self.import_csv_window())
    def on_close(self):
        self.save_data()
        self.save_config()
        self.destroy()
    def load_config(self):
        if os.path.exists(CONFIG_FILE):
            try:
```

```
            with open(CONFIG_FILE, "r", encoding="utf-8") as f:
                config = json.load(f)
                self.low_stock_threshold = config.get("low_stock_threshold", 10)
        except Exception:
            pass
    def save_config(self):
        try:
            with open(CONFIG_FILE, "w", encoding="utf-8") as f:
                json.dump({"low_stock_threshold": self.low_stock_threshold}, f, indent=4)
        except Exception:
            pass
    def load_data(self):
        if os.path.exists(FILE_NAME):
            try:
                with open(FILE_NAME, "r", encoding="utf-8") as f:
                    self.inventory = json.load(f)
                    for item in self.inventory.values():
                        self.categories.add(item.get("category", ""))
            except (json.JSONDecodeError, ValueError):
                messagebox.showwarning(
                    "Load Error",
                    "The inventory file is corrupted. Starting with an empty inventory."
                )
                self.inventory = {}
                self.categories = set()
    def save_data(self):
        try:
            with open(FILE_NAME, "w", encoding="utf-8") as f:
                json.dump(self.inventory, f, indent=4)
            self.update_summary()
        except OSError as exc:
            messagebox.showerror("Save Error", f"Could not save inventory:¥n{exc}")
    def import_csv_window(self):
        file_path = filedialog.askopenfilename(filetypes=[("CSV files", "*.csv"), ("All files", "*.*")], title="Import Inventory CSV")
        if not file_path:
            return
        try:
            with open(file_path, "r", encoding="utf-8") as f:
                reader = csv.DictReader(f)
                imported = 0
                for row in reader:
                    pid = row.get("ID") or row.get("id")
```

```
                if not pid:
                    continue
                if pid in self.inventory:
                    messagebox.showwarning("Skip", f"Product {pid} already exists, skipping.")
                    continue
                try:
                    self.inventory[pid] = {
                        "name": row.get("Name") or row.get("name") or "Unknown",
                        "category": row.get("Category") or row.get("category") or "",
                        "qty": int(row.get("Qty") or row.get("qty") or 0),
                        "price": float(row.get("Price") or row.get("price") or 0),
                        "supplier": row.get("Supplier") or row.get("supplier") or ""
                    }
                    cat = self.inventory[pid]["category"]
                    if cat:
                        self.categories.add(cat)
                    imported += 1
                except (ValueError, KeyError):
                    continue
            self.save_data()
            messagebox.showinfo("Imported", f"Successfully imported {imported} products.")
        except Exception as e:
            messagebox.showerror("Import Error", f"Failed to import CSV:¥n{e}")
    def export_inventory_shortcut(self):
        file_path = filedialog.asksaveasfilename(defaultextension=".csv", filetypes=[("CSV files", "*.csv"), ("All files", "*.*")], title="Export Inventory")
        if not file_path:
            return
        try:
            with open(file_path, "w", newline='', encoding='utf-8') as csvfile:
                writer = csv.writer(csvfile)
                writer.writerow(["ID", "Name", "Category", "Qty", "Price", "Supplier"])
                for pid, item in self.inventory.items():
                    writer.writerow([pid, item["name"], item["category"], item["qty"], item["price"], item["supplier"]])
            messagebox.showinfo("Exported", f"Exported to {os.path.basename(file_path)}")
        except Exception as e:
            messagebox.showerror("Export Error", f"Failed to export:¥n{e}")
    def create_header(self):
        header = ctk.CTkFrame(self, fg_color="#24315E", height=90, corner_radius=0)
        header.pack(fill="x")
        title = ctk.CTkLabel(
            header,
            text="🏭 Smart Inventory Management System",
```

```
            text_color="white",
            font=("Segoe UI", 30, "bold")
        )
        title.pack(pady=22)
    def create_footer(self):
        footer = ctk.CTkFrame(self, fg_color="#24315E", height=60, corner_radius=0)
        footer.pack(side="bottom", fill="x")
        label = ctk.CTkLabel(
            footer,
            text="Data Structures Used: dict | set | list | tuple",
            text_color="white",
            font=("Segoe UI", 14)
        )
        label.pack(side="left", padx=20, pady=10)
        ctk.CTkButton(footer, text="Settings", command=self.open_settings, width=90, fg_color="#3b82f6").pack(side="right", padx=6, pady=6)
        exit_button = ctk.CTkButton(
            footer,
            text="EXIT",
            fg_color="#ef4444",
            hover_color="#fca5a5",
            text_color="white",
            font=("Segoe UI", 14, "bold"),
            command=self.on_close,
            width=100
        )
        exit_button.pack(side="right", padx=20, pady=6)
    def open_settings(self):
        if getattr(self, "active_child_window", None) and self.active_child_window.winfo_exists():
            self.active_child_window.lift()
            return
        win = ctk.CTkToplevel(self)
        win.title("Settings")
        win.geometry("400x300")
        self.set_active_window(win)
        ctk.CTkLabel(win, text="Settings", font=("Segoe UI", 18, "bold")).pack(padx=20, pady=(20, 16), anchor="w")
        ctk.CTkLabel(win, text="Low Stock Threshold:", font=("Segoe UI", 12)).pack(padx=20, pady=(12, 4), anchor="w")
        threshold_var = ctk.StringVar(value=str(self.low_stock_threshold))
        ctk.CTkEntry(win, textvariable=threshold_var, width=300).pack(padx=20, pady=4)

        def save_settings():
            try:
                threshold = int(threshold_var.get())
```

```
            if threshold > 0:
                self.low_stock_threshold = threshold
                self.save_config()
                messagebox.showinfo("Success", "Settings saved successfully")
                self.close_active_window(win)
            else:
                messagebox.showerror("Error", "Threshold must be positive")
        except ValueError:
            messagebox.showerror("Error", "Please enter a valid number")

    ctk.CTkButton(win, text="Save Settings", command=save_settings, width=200).pack(pady=24)
def create_summary(self):
    summary_frame = ctk.CTkFrame(self, fg_color="#F8FAFC", corner_radius=22)
    summary_frame.pack(fill="x", padx=20, pady=(16, 4))
    self.summary_labels = {}
    metrics = [
        ("Total Products", "products"),
        ("Inventory Value", "value"),
        ("Low Stock", "low_stock"),
        ("Out Of Stock", "out_of_stock"),
        ("Categories", "categories")
    ]
    for text, key in metrics:
        card = ctk.CTkFrame(summary_frame, fg_color="white", corner_radius=18)
        card.pack(side="left", expand=True, fill="both", padx=10, pady=12)
        ctk.CTkLabel(card, text=text, text_color="#334155", font=("Segoe UI", 14)).pack(pady=(20, 6))
        label = ctk.CTkLabel(card, text="0", text_color="#0f172a", font=("Segoe UI", 26, "bold"))
        label.pack(pady=(0, 18))
        self.summary_labels[key] = label
def update_summary(self):
    if not hasattr(self, "summary_labels"):
        return
    total_products = len(self.inventory)
    total_value = sum(item["qty"] * item["price"] for item in self.inventory.values())
    low_stock = sum(1 for item in self.inventory.values() if item["qty"] < 10)
    out_of_stock = sum(1 for item in self.inventory.values() if item["qty"] == 0)
    categories = len(self.categories)
    self.summary_labels["products"].configure(text=str(total_products))
    self.summary_labels["value"].configure(text=f"₹{total_value:,.2f}")
    self.summary_labels["low_stock"].configure(text=str(low_stock))
    self.summary_labels["out_of_stock"].configure(text=str(out_of_stock))
    self.summary_labels["categories"].configure(text=str(categories))
```

```
        low_stock = sum(1 for item in self.inventory.values() if item["qty"] < self.low_stock_threshold)
        self.summary_labels["low_stock"].configure(text=str(low_stock))
    def create_dashboard_stats(self):
        self.body = ctk.CTkFrame(self, fg_color="#ECECEC")
        self.body.pack(fill="both", expand=True, padx=20, pady=20)

        cards = [
            ("Add Product", self.add_product_window),
            ("Update Product", self.update_product_window),
            ("Search Product", self.search_product_window),
            ("Display Inventory", self.display_inventory),
            ("Low Stock Alert", self.low_stock_alert),
            ("Out of Stock Alert", self.out_of_stock_alert),
            ("Category Management", self.show_categories),
            ("Inventory Report", self.inventory_report),
            ("Delete Product", self.delete_product_window)
        ]

        for col_index in range(3):
            self.body.grid_columnconfigure(col_index, weight=1, uniform="buttons")

        row = 0
        col = 0
        for text, command in cards:
            btn = ctk.CTkButton(
                self.body,
                text=text,
                height=110,
                font=("Segoe UI", 22, "bold"),
                fg_color="white",
                text_color="#1E293B",
                hover_color="#dfe7fd",
                corner_radius=18,
                command=command
            )
            btn.grid(row=row, column=col, padx=12, pady=12, sticky="nsew")
            self.body.grid_rowconfigure(row, weight=1)
            col += 1
            if col > 2:
                col = 0
                row += 1
    def toggle_dashboard_buttons(self, enable: bool):
```

```
252         if not hasattr(self, "body"):
253             return
254         for child in self.body.winfo_children():
255             if isinstance(child, ctk.CTkButton):
256                 child.configure(state="normal" if enable else "disabled")
257     def set_active_window(self, win):
258         self.active_child_window = win
259         self.toggle_dashboard_buttons(False)
260         win.transient(self)
261         win.grab_set()
262         win.lift()
263         win.focus_force()
264         win.protocol("WM_DELETE_WINDOW", lambda w=win: self.close_active_window(w))
265     def close_active_window(self, win):
266         if getattr(self, "active_child_window", None) is win:
267             self.active_child_window = None
268         self.toggle_dashboard_buttons(True)
269         try:
270             win.grab_release()
271         except Exception:
272             pass
273         win.destroy()
274     def validate_product_fields(self, fields):
275         if not fields["Product ID"]:
276             return "Product ID is required."
277         if not fields["Name"]:
278             return "Product name is required."
279         if not fields["Category"]:
280             return "Category is required."
281         try:
282             qty = int(fields["Quantity"])
283             if qty < 0:
284                 return "Quantity cannot be negative."
285         except ValueError:
286             return "Quantity must be a whole number."
287         try:
288             price = float(fields["Price"])
289             if price < 0:
290                 return "Price cannot be negative."
291         except ValueError:
292             return "Price must be a number."
293         if not fields["Supplier"]:
```

```
            return "Supplier is required."
        return ""
    def add_product_window(self):
        if getattr(self, "active_child_window", None) and self.active_child_window.winfo_exists():
            self.active_child_window.lift()
            self.active_child_window.focus_force()
            return
        win = ctk.CTkToplevel(self)
        win.title("Add Product")
        win.geometry("480x680")
        self.set_active_window(win)
        entries = {}
        fields = ["Product ID", "Name", "Category", "Quantity", "Price", "Supplier", "Supplier Email", "Supplier Phone"]
        for field in fields:
            ctk.CTkLabel(win, text=field, anchor="w").pack(fill="x", padx=20, pady=(12, 4))
            ent = ctk.CTkEntry(win, width=420)
            ent.pack(padx=20)
            entries[field] = ent
        if self.categories:
            category_hint = ", ".join(sorted(self.categories))
            ctk.CTkLabel(
                win,
                text=f"Existing categories: {category_hint}",
                text_color="#475569",
                font=("Segoe UI", 11),
                anchor="w"
            ).pack(fill="x", padx=20, pady=(8, 0))
        def save():
            values = {field: entries[field].get().strip() for field in fields}
            validation_error = self.validate_product_fields(values)
            if validation_error:
                messagebox.showerror("Invalid Input", validation_error)
                return
            pid = values["Product ID"]
            if pid in self.inventory:
                messagebox.showerror("Error", "Product already exists")
                return
            self.inventory[pid] = {
                "name": values["Name"],
                "category": values["Category"],
                "qty": int(values["Quantity"]),
                "price": float(values["Price"]),
```

```
            "supplier": values["Supplier"],
            "supplier_email": values["Supplier Email"],
            "supplier_phone": values["Supplier Phone"]
        }
        self.categories.add(values["Category"])
        self.save_data()
        messagebox.showinfo("Success", "Product added successfully")
        self.close_active_window(win)
    ctk.CTkButton(win, text="Save Product", command=save, width=200).pack(pady=24)
def display_inventory(self):
    if getattr(self, "active_child_window", None) and self.active_child_window.winfo_exists():
        self.active_child_window.lift()
        self.active_child_window.focus_force()
        return
    win = ctk.CTkToplevel(self)
    win.title("Inventory Table")
    win.geometry("1140x620")
    self.set_active_window(win)

    top_frame = ctk.CTkFrame(win)
    top_frame.pack(fill="x", padx=12, pady=(12, 6))
    ctk.CTkLabel(top_frame, text="Filter:", width=70).pack(side="left", padx=(8, 6))
    filter_var = ctk.StringVar()
    filter_entry = ctk.CTkEntry(top_frame, textvariable=filter_var, width=360)
    filter_entry.pack(side="left", padx=(0, 8))
    def on_export():
        try:
            with open("inventory_export.csv", "w", newline='', encoding='utf-8') as csvfile:
                writer = csv.writer(csvfile)
                writer.writerow(["ID", "Name", "Category", "Qty", "Price", "Supplier"])
                for pid, item in self.inventory.items():
                    writer.writerow([pid, item["name"], item["category"], item["qty"], item["price"], item["supplier"]])
            messagebox.showinfo("Exported", "Exported to inventory_export.csv")
        except Exception as e:
            messagebox.showerror("Error", f"Export failed: {e}")
    def on_bulk_delete():
        selected = tree.selection()
        if not selected:
            messagebox.showwarning("No Selection", "Please select products to delete")
            return
        pids = [tree.item(iid, 'values')[0] for iid in selected]
        confirm = messagebox.askyesno("Confirm Bulk Delete", f"Delete {len(pids)} selected products?")
```

```
            if confirm:
                for pid in pids:
                    del self.inventory[pid]
                self.save_data()
                populate_tree()
                messagebox.showinfo("Deleted", f"Deleted {len(pids)} products")
        ctk.CTkButton(top_frame, text="Export CSV", command=on_export, width=110).pack(side="right", padx=8)
        ctk.CTkButton(top_frame, text="Delete Selected", command=on_bulk_delete, width=120, fg_color="#ef4444").pack(side="right", padx=8)
        ctk.CTkButton(top_frame, text="Refresh", command=lambda: populate_tree(), width=110).pack(side="right")

        container = ctk.CTkFrame(win)
        container.pack(fill="both", expand=True, padx=12, pady=6)
        columns = ("ID", "Name", "Category", "Qty", "Price", "Supplier")

        tree = ttk.Treeview(container, columns=columns, show="headings", selectmode="extended")
        for col in columns:
            tree.heading(col, text=col)
            tree.column(col, width=160, anchor="center")
        tree.pack(fill="both", expand=True)
        def sort_tree(col, reverse=False):
            l = [(tree.set(k, col), k) for k in tree.get_children('')]
            try:
                l.sort(key=lambda t: float(t[0].replace('₹','').replace(',','')), reverse=reverse)
            except Exception:
                l.sort(key=lambda t: t[0].lower(), reverse=reverse)
            for index, (val, k) in enumerate(l):
                tree.move(k, '', index)
            tree.heading(col, command=lambda: sort_tree(col, not reverse))
        for col in columns:
            tree.heading(col, text=col, command=lambda c=col: sort_tree(c, False))

        def on_double_click(event):
            item_id = tree.selection()
            if not item_id:
                return
            vals = tree.item(item_id, 'values')
            pid = vals[0]
            self.edit_product_window(pid)

        tree.bind('<Double-1>', on_double_click)

        def populate_tree():
```

```
        q = filter_var.get().strip().lower()
        for i in tree.get_children():
            tree.delete(i)
        for idx, (pid, item) in enumerate(self.inventory.items()):
            if q:
                if q not in pid.lower() and q not in item['name'].lower() and q not in item['category'].lower():
                    continue
            tag = "oddrow" if idx % 2 else "evenrow"
            tree.insert('', 'end', values=(pid, item['name'], item['category'], item['qty'], f"₹{item['price']:.2f}", item['supplier']), tags=(tag,))
        tree.tag_configure("oddrow", background="#f7f9ff")
        tree.tag_configure("evenrow", background="white")
    filter_entry.bind('<KeyRelease>', lambda e: populate_tree())
    populate_tree()
def search_product_window(self):
    search_win = ctk.CTkToplevel(self)
    search_win.title("Search Products")
    search_win.geometry("500x280")
    search_win.transient(self)
    search_win.grab_set()

    ctk.CTkLabel(search_win, text="Search by:", font=("Segoe UI", 12, "bold")).pack(padx=20, pady=(16, 8), anchor="w")
    query_var = ctk.StringVar()
    ctk.CTkEntry(search_win, placeholder_text="ID, name, or category...", textvariable=query_var, width=440).pack(padx=20, pady=4)
    ctk.CTkLabel(search_win, text="Price Range:", font=("Segoe UI", 12, "bold")).pack(padx=20, pady=(12, 8), anchor="w")
    price_frame = ctk.CTkFrame(search_win)
    price_frame.pack(padx=20, pady=4, fill="x")
    ctk.CTkLabel(price_frame, text="₹", width=30).pack(side="left")
    min_price = ctk.CTkEntry(price_frame, placeholder_text="Min", width=150)
    min_price.pack(side="left", padx=(0, 8))
    ctk.CTkLabel(price_frame, text="to ₹", width=40).pack(side="left")
    max_price = ctk.CTkEntry(price_frame, placeholder_text="Max", width=150)
    max_price.pack(side="left")

    def search():
        query = query_var.get().strip().lower()
        min_p = float(min_price.get()) if min_price.get().strip() else 0
        max_p = float(max_price.get()) if max_price.get().strip() else float('inf')
        results = []
        if query:
            if query in self.inventory:
                results.append(self.inventory[query])
            for pid, item in self.inventory.items():
```

```
                if query in pid.lower() or query in item["name"].lower() or query in item["category"].lower():
                    if item not in results:
                        results.append(item)
            else:
                results = list(self.inventory.values())
            results = [item for item in results if min_p <= item["price"] <= max_p]
            if not results:
                messagebox.showinfo("Search Results", "No products match your criteria.")
                return
            formatted = [f"ID: {pid}¥nName: {item['name']}¥nCategory: {item['category']}¥nQty: {item['qty']}¥nPrice: ₹{item['price']:,.2f}¥nSupplier: {item['supplier']}" for pid, item in [(pid, item) for pid, item in self.inventory.items() if item in results]]
            messagebox.showinfo("Search Results", "¥n¥n".join(formatted))
        ctk.CTkButton(search_win, text="Search", command=search, width=200).pack(pady=16)
    def delete_product_window(self):
        pid = ctk.CTkInputDialog(text="Enter Product ID:", title="Delete Product").get_input()
        if not pid:
            return
        if pid in self.inventory:
            confirm = messagebox.askyesno("Confirm Delete", f"Delete product {pid}? This cannot be undone.")
            if confirm:
                del self.inventory[pid]
                self.save_data()
                messagebox.showinfo("Deleted", "Product deleted")
        else:
            messagebox.showerror("Error", "Product not found")
    def edit_product_window(self, pid):
        if getattr(self, "active_child_window", None) and self.active_child_window.winfo_exists():
            self.active_child_window.lift()
            self.active_child_window.focus_force()
            return
        if pid not in self.inventory:
            messagebox.showerror("Error", "Product not found")
            return
        item = self.inventory[pid]
        win = ctk.CTkToplevel(self)
        win.title(f"Update Product - {pid}")
        win.geometry("480x680")
        self.set_active_window(win)
        entries = {}
        fields = ["Name", "Category", "Quantity", "Price", "Supplier", "Supplier Email", "Supplier Phone"]
        defaults = {
            "Name": item["name"],
```

```
            "Category": item["category"],
            "Quantity": str(item["qty"]),
            "Price": str(item["price"]),
            "Supplier": item["supplier"],
            "Supplier Email": item.get("supplier_email", ""),
            "Supplier Phone": item.get("supplier_phone", "")
        }
        for field in fields:
            ctk.CTkLabel(win, text=field, anchor="w").pack(fill="x", padx=20, pady=(12, 4))
            ent = ctk.CTkEntry(win, width=420)
            ent.insert(0, defaults[field])
            ent.pack(padx=20)
            entries[field] = ent
        if self.categories:
            category_hint = ", ".join(sorted(self.categories))
            ctk.CTkLabel(
                win,
                text=f"Existing categories: {category_hint}",
                text_color="#475569",
                font=("Segoe UI", 11),
                anchor="w"
            ).pack(fill="x", padx=20, pady=(8, 0))
        def save_update():
            values = {field: entries[field].get().strip() for field in fields}
            missing = [f for f, v in values.items() if not v]
            if missing:
                messagebox.showerror("Invalid Input", f"Please fill: {', '.join(missing)}")
                return
            validation_error = self.validate_product_fields({
                "Product ID": pid,
                **values
            })
            if validation_error:
                messagebox.showerror("Invalid Input", validation_error)
                return
            self.inventory[pid].update({
                "name": values["Name"],
                "category": values["Category"],
                "qty": int(values["Quantity"]),
                "price": float(values["Price"]),
                "supplier": values["Supplier"],
                "supplier_email": values["Supplier Email"],
```

```
            "supplier_phone": values["Supplier Phone"]
        })
        self.categories.add(values["Category"])
        self.save_data()
        messagebox.showinfo("Updated", "Product updated successfully")
        self.close_active_window(win)
    ctk.CTkButton(win, text="Save Changes", command=save_update, width=200).pack(pady=24)
def update_product_window(self):
    if getattr(self, "active_child_window", None) and self.active_child_window.winfo_exists():
        self.active_child_window.lift()
        self.active_child_window.focus_force()
        return
    pid = ctk.CTkInputDialog(text="Enter Product ID:", title="Update Product").get_input()
    if not pid:
        return
    self.edit_product_window(pid)
def low_stock_alert(self):
    low = [f"{pid} - {item['name']}" for pid, item in self.inventory.items() if item["qty"] < self.low_stock_threshold]
    result = messagebox.showinfo("Low Stock Alert", f"Threshold: {self.low_stock_threshold} units¥n¥n" + ("¥n".join(low) if low else "No low stock items"))
def out_of_stock_alert(self):
    out = [f"{pid} - {item['name']}" for pid, item in self.inventory.items() if item["qty"] == 0]
    messagebox.showinfo("Out Of Stock", "¥n".join(out) if out else "No out-of-stock items")
def show_categories(self):
    if getattr(self, "active_child_window", None) and self.active_child_window.winfo_exists():
        self.active_child_window.lift()
        self.active_child_window.focus_force()
        return
    win = ctk.CTkToplevel(self)
    win.title("Category Management")
    win.geometry("520x420")
    self.set_active_window(win)

    top = ctk.CTkFrame(win)
    top.pack(fill="x", padx=12, pady=12)
    new_cat_var = ctk.StringVar()
    ctk.CTkEntry(top, textvariable=new_cat_var, placeholder_text="New category name...").pack(side="left", padx=(6,8), fill="x", expand=True)
    def add_category():
        name = new_cat_var.get().strip()
        if not name:
            return
        if name in self.categories:
            messagebox.showerror("Error", "Category already exists")
```

```
            return
        self.categories.add(name)
        self.save_data()
        refresh()
        new_cat_var.set("")
    ctk.CTkButton(top, text="Add", command=add_category, width=90).pack(side="right", padx=6)
    list_frame = ctk.CTkFrame(win)
    list_frame.pack(fill="both", expand=True, padx=12, pady=(0, 12))

    tree = ttk.Treeview(list_frame, columns=("cat", "count"), show="headings", height=12)
    tree.heading("cat", text="Category")
    tree.heading("count", text="# Items")
    tree.column("cat", width=320)
    tree.column("count", width=80, anchor="center")
    tree.pack(fill="both", expand=True)

    def refresh():
        for i in tree.get_children():
            tree.delete(i)
        counts = {}
        for pid, item in self.inventory.items():
            counts[item.get('category','')] = counts.get(item.get('category',''), 0) + 1
        for cat in sorted(self.categories):
            tree.insert('', 'end', values=(cat, counts.get(cat, 0)))

    def delete_selected():
        sel = tree.selection()
        if not sel:
            return
        cat = tree.item(sel[0], 'values')[0]
        confirm = messagebox.askyesno("Confirm", f"Delete category '{cat}'? Products with this category will be set to empty.")
        if not confirm:
            return
        for pid, item in list(self.inventory.items()):
            if item.get('category') == cat:
                self.inventory[pid]['category'] = ''
        if cat in self.categories:
            self.categories.remove(cat)
        self.save_data()
        refresh()
    btn_frame = ctk.CTkFrame(win)
    btn_frame.pack(fill="x", padx=12, pady=(0,12))
```

```
        ctk.CTkButton(btn_frame, text="Delete Selected", command=delete_selected, width=140, fg_color="#ef4444").pack(side="right", padx=6)
        refresh()
    def inventory_report(self):
        if getattr(self, "active_child_window", None) and self.active_child_window.winfo_exists():
            self.active_child_window.lift()
            self.active_child_window.focus_force()
            return
        if not self.inventory:
            messagebox.showinfo("Inventory Report", "No inventory data available.")
            return

        win = ctk.CTkToplevel(self)
        win.title("Inventory Report")
        win.geometry("980x620")
        self.set_active_window(win)

        total_items = len(self.inventory)
        total_qty = sum(item["qty"] for item in self.inventory.values())
        total_value = sum(item["qty"] * item["price"] for item in self.inventory.values())

        header = ctk.CTkFrame(win, fg_color="#F8FAFC", corner_radius=18)
        header.pack(fill="x", padx=16, pady=(16, 8))
        ctk.CTkLabel(
            header,
            text="Inventory Report",
            font=("Segoe UI", 24, "bold"),
            text_color="#1F2937"
        ).pack(anchor="w", padx=16, pady=(16, 8))
        ctk.CTkLabel(
            header,
            text=(
                f"Products: {total_items}    "
                f"Total Quantity: {total_qty}    "
                f"Total Value: ₹{total_value:,.2f}    "
                f"Categories: {len(self.categories)}"
            ),
            font=("Segoe UI", 12),
            text_color="#475569"
        ).pack(anchor="w", padx=16, pady=(0, 16))
        data = [
            (pid, item["name"], item["qty"], item["qty"] * item["price"])
            for pid, item in self.inventory.items()
```

```
        ]
        data.sort(key=lambda x: x[2], reverse=True)
        display_data = data[:8]
        chart_frame = ctk.CTkFrame(win, fg_color="#FFFFFF", corner_radius=18)
        chart_frame.pack(fill="both", expand=True, padx=16, pady=(0, 16))
        legend = ctk.CTkFrame(chart_frame, fg_color="#F8FAFC", corner_radius=14)
        legend.pack(fill="x", padx=16, pady=(16, 8))
        ctk.CTkLabel(legend, text="Legend:", font=("Segoe UI", 12, "bold"), text_color="#1F2937").pack(side="left", padx=(12, 8), pady=10)
        ctk.CTkLabel(legend, text="Quantity", font=("Segoe UI", 12), text_color="#2563EB").pack(side="left", padx=(0, 16), pady=10)
        ctk.CTkLabel(legend, text="Valuation", font=("Segoe UI", 12), text_color="#10B981").pack(side="left", padx=(0, 16), pady=10)
        canvas = Canvas(chart_frame, bg="#FFFFFF", highlightthickness=0)
        canvas.pack(fill="both", expand=True, padx=16, pady=16)

        chart_width = 900
        row_height = 45
        quantity_max = max(item[2] for item in display_data) or 1
        value_max = max(item[3] for item in display_data) or 1
        qty_bar_max = 300
        val_bar_max = 300
        qty_x = 140
        val_x = qty_x + qty_bar_max + 80

        for index, (_, name, qty, val) in enumerate(display_data):
            y = 24 + index * row_height
            canvas.create_text(16, y + 8, anchor="nw", text=name, font=("Segoe UI", 10, "bold"), fill="#111827")
            qty_width = int(qty_bar_max * qty / quantity_max)
            canvas.create_rectangle(qty_x, y, qty_x + qty_width, y + 20, fill="#2563EB", outline="")
            canvas.create_text(qty_x + qty_width + 8, y + 10, anchor="w", text=str(qty), font=("Segoe UI", 9), fill="#2563EB")
            val_width = int(val_bar_max * val / value_max)
            canvas.create_rectangle(val_x, y, val_x + val_width, y + 20, fill="#10B981", outline="")
            canvas.create_text(val_x + val_width + 8, y + 10, anchor="w", text=f"₹{val:,.0f}", font=("Segoe UI", 9), fill="#047857")

        if len(data) > 8:
            other_qty = sum(item[2] for item in data[8:])
            other_val = sum(item[3] for item in data[8:])
            y = 24 + len(display_data) * row_height
            canvas.create_text(16, y + 8, anchor="nw", text="Others", font=("Segoe UI", 10, "bold"), fill="#111827")
            qty_width = int(qty_bar_max * other_qty / quantity_max)
            canvas.create_rectangle(qty_x, y, qty_x + qty_width, y + 20, fill="#2563EB", outline="")
            canvas.create_text(qty_x + qty_width + 8, y + 10, anchor="w", text=str(other_qty), font=("Segoe UI", 9), fill="#2563EB")
            val_width = int(val_bar_max * other_val / value_max)
            canvas.create_rectangle(val_x, y, val_x + val_width, y + 20, fill="#10B981", outline="")
```

```
            canvas.create_text(val_x + val_width + 8, y + 10, anchor="w", text=f"₹{other_val:,.0f}", font=("Segoe UI", 9), fill="#047857")

if __name__ == "__main__":
    app = InventoryApp()
    app.mainloop()

```